

| Codebase | LLM Model                          | Function Name    | Bug Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------|------------------------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Libcsv   | GPT-4o, GPT-3.5-turbo, GPT-4o-mini | csv_write2       | This function writes the source character sequence into a buffer. In Rust, it checks if the output buffer is empty. It will return 0 directly if that is <i>empty</i> . But the original C code checks if the buffer is <i>NULL</i> and still return the length of the input string if the buffer is <i>NULL</i> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Libcsv   | GPT-4o                             | csv_set_blk_size | This function set <code>blk_size</code> field into the <code>csv_parser</code> struct. In Rust, if the <code>blk_size</code> field is 0, it will not assign the input value to the field. But in the C code, it will still assign the input value to field <code>blk_size</code> even the original value of <code>blk_size</code> is 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Libcsv   | GPT-4o                             | csv_set_delim    | This function set <code>delim_char</code> field into the <code>csv_parser</code> struct. In Rust, if the <code>delim_char</code> field is <code>'\0'</code> , it will not assign the input value to the field. But in the C code, it will still assign the input value to field <code>delim_char</code> even the original value of <code>delim_char</code> is <code>'\0'</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Libcsv   | GPT-3.5-turbo                      | csv_free         | This function frees the <code>char * field entry_buf</code> inside <code>csv_parser</code> and sets <code>entry_size</code> to 0. In Rust, if <code>entry_buf</code> is <i>NULL</i> , it does not set <code>entry_size</code> to 0, whereas the original C code still does.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| optipng  | Claude-3.5-Sonnet, GPT-4o-mini     | scan_option      | This function parses configuration options in the form <code>-option=value</code> . In the C code, the for loop first scans the option name. Then, if it encounters a <code>=</code> character indicating the separation between the option name and the value, the C code increments the <code>ptr</code> pointer to consume and discard the <code>=</code> symbol, and then stores the value to the <code>opt_arg_ptr</code> pointer. In the Rust code the next character is consumed at the end of the iteration which results in the <code>=</code> symbol being appended to the option name, instead of being discarded. Therefore, for an input in the form <code>option=value</code> , Rust will return <code>option=</code> as the option name.                                                                                                                                                                                                                                                                                                                                                                                                             |
| u8c      | GPT-4o                             | u8encode_        | This function converts a single Unicode code-point ( <code>ch</code> ) into its UTF-8 byte sequence, stores those bytes at the address <code>s</code> and use null to terminates the sequence, returns the number of bytes written (or -1 if <code>ch</code> lies outside the handled ranges). In the Rust version, it terminates the slice <code>s</code> by <code>s[len as usize]</code> and <code>len</code> is the length of the sequence. If <code>ch</code> is out of Unicode range, none of the else if branches run, so <code>len</code> remains -1. The final write uses <code>s[len as usize]</code> , which becomes <code>s[usize::MAX]</code> —an immediate out-of-bounds index that Rust traps with a panic.                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| u8c      | GPT-4o, GPT-4o-mini                | u8next_          | The original C function takes as input arguments UTF-8-encoded string <code>*txt</code> and another string <code>*ch</code> . It returns the number of bytes encoding the first Unicode codepoint in <code>txt</code> and stores converted unicode value in <code>ch</code> . If the encoding is not valid, it returns -1 as the return value and again stores the first byte of the string in <code>ch</code> . In the generated Rust code, When the input is not a valid UTF string, for example, a 2 character sequence such as <code>0xC2, 0x41</code> , the original C program will return the original first byte which is <code>0xC2</code> . But the transpiled Rust program will an intermediate state of decoding, specifically, the bit-masked value <code>0x02</code> .                                                                                                                                                                                                                                                                                                                                                                                 |
| u8c      | GPT-4o-mini                        | u8next_FAST      | This function takes as input arguments UTF-8-encoded string <code>*txt</code> and another string <code>*ch</code> . It returns the number of bytes encoding the first Unicode codepoint in <code>txt</code> and stores the converted unicode value in <code>ch</code> . In the C version the pointer expression <code>++s</code> walks the raw UTF-8 byte stream one byte at a time, so for <code>"€"</code> it reads the three bytes <code>E2, 82, AC</code> , applies the usual bit-masks ( <code>0xF0, 0xE0, 0x1F ...</code> ) to those individual bytes, and reconstructs the correct code-point <code>0x20AC</code> . The Rust port replaces that with <code>chars.next()</code> , which iterates over already-decoded Unicode scalars—the first call instantly yields the whole character <code>'€'</code> (value <code>0x20AC</code> ) and there are no continuation bytes left to inspect. When the code then runs the same byte-oriented masks on this full scalar and tries to splice in the “next” character, the bit pattern is mis-aligned, so the assembled value becomes the garbage code-point <code>0x0321</code> instead of <code>0x20AC</code> . |
| u8c      | Claude-3.5-Sonnet                  | u8next_FAST      | This function takes as input arguments UTF-8-encoded string <code>*txt</code> and another string <code>*ch</code> . It returns the number of bytes encoding the first Unicode codepoint in <code>txt</code> and stores the converted unicode value in <code>ch</code> . When the input string is empty (i.e., its first byte is <code>'\0'</code> in C or the byte slice has length 0 in Rust), both implementations report a length of 0, but they differ in what they do with the output-parameter for the decoded code-point. C version ( <code>u8next_FAST</code> ) writes 0 to <code>*ch</code> explicitly signalling “no character.” Rust version ( <code>u8next_fast</code> ) enters its body only if <code>s.get(0)</code> succeeds; Consequently, the function leaves the <code>ch</code> reference untouched (whatever value the caller held remains), and only returns 0.                                                                                                                                                                                                                                                                                |

TABLE VI: Bugs detected by RustAssure